

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»
імені ІГОРЯ СІКОРСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ
КАФЕДРА МАТЕМАТИЧНИХ МЕТОДІВ ЗАХИСТУ ІНФОРМАЦІЇ

ЗВІТ
з лабораторної роботи № 5
з теми « Робота з базовими функціями БД типу column family на прикладі
Cassandra»

Виконав:
студент ФБ-
41мн
Дрик Владислав
Олександрович
«10» Жовтня 2024
р

КИЇВ 2024

Частина 1. Робота зі структурами даних у Cassandra

Завдання:

У таблиці *items* містить різноманітні товари (тобто у яких різний набір властивостей).

Для набору властивостей товару виберіть базові характеристики однакові для всіх товарів (назва, категорія, ціна, виробник, ...), а для властивостей які відрізняються використовуйте тип *map* (з індексом для можливості пошуку по її вмісту)

Необхідно, щоб пошук швидко працював для *категорії* товарів. Ця вимога має бути врахована при створенні ключа для таблиці.

```
4 usages
@Table("items")
public class Item {
    4 usages
    @PrimaryKey
    private UUID id;
    4 usages
    private String name;
    4 usages
    private String category;
    4 usages
    private BigDecimal price;
    4 usages
    private String manufacturer;
    4 usages
    private Map<String, String> additionalProperties;

    6 usages
    public Item() {}

    6 usages
    public Item(UUID id, String name, String category, BigDecimal price, String manufacturer, Map<String, String> additionalProperties) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.price = price;
        this.manufacturer = manufacturer;
        this.additionalProperties = additionalProperties;
    }
}
```

!!! У запитах заборонено використовувати *ALLOW FILTERING* !!!

1. Напишіть запит, який показує структуру створеної таблиці (команда *DESCRIBE*)

`DESCRIBE my_keyspace.items;`

2. Напишіть запит, який виводить усі товари в певній категорії відсортовані за ціною

```
public interface ItemRepository extends CassandraRepository<Item, UUID> {

    no usages
    @Query("SELECT * FROM my_keyspace.items WHERE category = ?0 ORDER BY price ASC")
    List<Item> findByCategoryOrderByPrice(String category);

}
```

3. Напишіть запити, які вибирають товари за різними критеріями в межах певної категорії (тут де треба замість індексу використайте Materialized view):
- а. назва,
 - б. ціна (в проміжку),
 - в. ціна та виробник

```
no usages
@Query("SELECT * FROM my_keyspace.by_name WHERE category = ?0 AND name = ?1")
List<Item> findByNameAndCategory(String category, String name);

no usages
@Query("SELECT * FROM my_keyspace.by_price_range WHERE category = ?0 AND price >= ?1 AND price <= ?2")
List<Item> findByPriceRangeAndCategory(String category, BigDecimal minPrice, BigDecimal maxPrice);

no usages
@Query("SELECT * FROM my_keyspace.by_price_and_manufacturer WHERE category = ?0 AND price = ?1 AND manufacturer = ?2")
List<Item> findByPriceAndManufacturerAndCategory(String category, BigDecimal price, String manufacturer);
```

Матеріалізований вигляд для вибору за назвою:

```
CREATE MATERIALIZED VIEW IF NOT EXISTS my_keyspace.by_name AS
SELECT category, price, manufacturer, id, name
FROM my_keyspace.items
WHERE name IS NOT NULL
PRIMARY KEY (name, category);
```

Матеріалізований вигляд для вибору за ціною в проміжку:

```
CREATE MATERIALIZED VIEW IF NOT EXISTS my_keyspace.by_price_range AS
SELECT category, price, manufacturer, id, name
FROM my_keyspace.items
WHERE price IS NOT NULL AND category IS NOT NULL
PRIMARY KEY (category, price);
```

Матеріалізований вигляд для вибору за ціною та виробником:

```
CREATE MATERIALIZED VIEW IF NOT EXISTS my_keyspace.by_price_and_manufacturer AS
SELECT category, price, manufacturer, id, name
FROM my_keyspace.items
WHERE price IS NOT NULL AND manufacturer IS NOT NULL
PRIMARY KEY (manufacturer, price);
```

Створіть таблицю *orders* в якій міститься ім'я замовника і інформація про замовлення: перелік id-товарів у замовленні, вартість замовлення, дата замовлення,

```
@Table("orders")
public class Order {
    4 usages
    @PrimaryKey
    private UUID id;
    4 usages
    private UUID itemId;
    4 usages
    private String customerName;
    4 usages
    private String orderDate;
    4 usages
    private int quantity;

    no usages
    public Order(){}

    no usages
    public Order(UUID id, UUID itemId, String customerName, String orderDate, int quantity) {
        this.id = id;
        this.itemId = itemId;
        this.customerName = customerName;
        this.orderDate = orderDate;
        this.quantity = quantity;
    }
}
```

Для кожного замовника повинна бути можливість швидко шукати його замовлення і виконувати по них запити. Ця вимога має бути врахована при створенні ключа для таблиці.

1. Напишіть запит, який показує структуру створеної таблиці (команда *DESCRIBE*)

`DESCRIBE my_keyspace.orders;`

2. Для замовника виведіть всі його замовлення відсортовані за часом коли вони були зроблені
3. Для кожного замовників визначте суму на яку були зроблені усі його замовлення
4. Для кожного замовлення виведіть час коли його ціна були занесена в базу (`SELECT WRITETIME`)

```

public interface OrderRepository extends CassandraRepository<Order, UUID> {

    no usages
    @Query("SELECT * FROM my_keyspace.orders WHERE customer_name = ?0 ORDER BY order_date DESC")
    List<Order> findByCustomerNameOrderByOrderDateDesc(String customerName);

    no usages
    @Query("SELECT SUM(total_price) FROM my_keyspace.orders WHERE customer_name = ?0")
    BigDecimal findTotalAmountByCustomerName(String customerName);

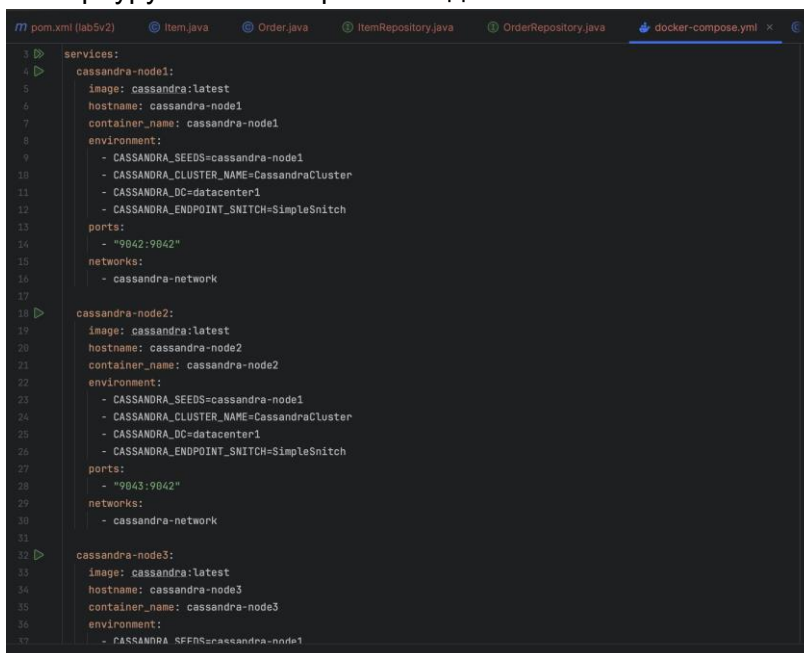
    no usages
    @Query("SELECT WRITETIME(total_price) FROM my_keyspace.orders WHERE id = ?0")
    Long findWriteTimeByOrderId(UUID orderId);
}

```

Частина 2. Налаштування реплікації у Cassandra

Завдання

1. Сконфігурувати кластер з 3-х нод:



```

1  services:
2    cassandra-node1:
3      image: cassandra:latest
4      hostname: cassandra-node1
5      container_name: cassandra-node1
6      environment:
7        - CASSANDRA_SEEDS=cassandra-node1
8        - CASSANDRA_CLUSTER_NAME=CassandraCluster
9        - CASSANDRA_DC=datacenter1
10       - CASSANDRA_ENDPOINT_SNITCH=SimpleSnitch
11      ports:
12        - "9042:9042"
13      networks:
14        - cassandra-network
15
16    cassandra-node2:
17      image: cassandra:latest
18      hostname: cassandra-node2
19      container_name: cassandra-node2
20      environment:
21        - CASSANDRA_SEEDS=cassandra-node1
22        - CASSANDRA_CLUSTER_NAME=CassandraCluster
23        - CASSANDRA_DC=datacenter1
24        - CASSANDRA_ENDPOINT_SNITCH=SimpleSnitch
25      ports:
26        - "9043:9042"
27      networks:
28        - cassandra-network
29
30    cassandra-node3:
31      image: cassandra:latest
32      hostname: cassandra-node3
33      container_name: cassandra-node3
34      environment:
35        - CASSANDRA_SEEDS=cassandra-node1

```

```
(base) prosto@MacBook-Pro-Malchik lab5v2 % docker compose up -d
WARN[0000] /Users/prosto/Documents/highload_systems/lab5v2/docker-com
tial confusion
[+] Running 13/3
  ✓ cassandra-node1 Pulled
  ✓ cassandra-node2 Pulled
  ✓ cassandra-node3 Pulled
[+] Running 4/4
  ✓ Network lab5v2_cassandra-network Created
  ✓ Container cassandra-node2 Started
  ✓ Container cassandra-node1 Started
  ✓ Container cassandra-node3 Started
(base) prosto@MacBook-Pro-Malchik lab5v2 %
```

Container CPU usage ⓘ

14.08% / 800% (8 CPUs available)

Container memory usage ⓘ

7.03GB / 7.09GB

Show charts

Q

Search

Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	▼ lab5v2	-	-	-	14.08%	3 minutes ago	
☐	cassandra-node1	cb100f8a4c4f	cassandra:latest	9042:9042 ↗	6.05%	3 minutes ago	
☐	cassandra-node3	53c6fe67caa1	cassandra:latest	9044:9042 ↗	4.57%	3 minutes ago	
☐	cassandra-node2	80e8f53637a2	cassandra:latest	9043:9042 ↗	3.46%	3 minutes ago	

2. Перевірити правильність конфігурації за допомогою

nodetool status

```
# nodetool status
Datacenter: my-datacenter-1
=====
Status=Up/Down
// State=Normal/Leaving/Joining/Moving
-- Address      Load        Tokens      Owns (effective)  Host ID                               Rack
UN  172.18.0.3    80.05 KiB   16          59.3%             8a216e09-224e-402b-b392-5ef303c895b8 rack1
UN  172.18.0.2    119.84 KiB  16          64.7%             cc595379-7f87-4539-a01f-8100e098629c rack1
UN  172.18.0.4    85.1 KiB   16          76.0%             a0a5040e-3aa6-4a31-b021-9cd6081c1624 rack1
```

3. Використовуючи *cqlsh*, створити три *Keyspace* з replication factor 1, 2, 3 з SimpleStrategy

```
CREATE KEYSPACE IF NOT EXISTS keyspace_name
```

```
WITH REPLICATION = {
```

```
    'class': 'SimpleStrategy',
```

```
    'replication_factor': 3
```

```
};
```

```
# nodetool status
Datacenter: datacenter1
=====
Status=Up/Down
// State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens    Owns (effective)  Host ID                               Rack
UN 172.19.0.4    185.99 KiB    16        32.7%             929fc404-7451-4ed1-b161-23d0f7586a9d rack1
UN 172.19.0.2    147.99 KiB    16        35.7%             14c7589a-d829-4a37-bb6c-8a2ef9824bb8 rack1
UN 172.19.0.3    174.24 KiB    16        31.6%             486a0aed-5f84-4dce-9576-35187ce21b18 rack1
```

4. В кожному з кейспейсів створити прості таблиці

```
CREATE TABLE IF NOT EXISTS orders.items_by_category (
```

```
    id INT,
```

```
    name TEXT,
```

```
    category TEXT,
```

```
    price DECIMAL,
```

```
    manufacturer TEXT,
```

```
    short_description TEXT,
```

```
    custom_attributes MAP<TEXT, TEXT>,
```

```
    PRIMARY KEY ((category), price, id)
```

```
);
```

```
# cqlsh
Connected to CassandraCluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.2 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> CREATE KEYSPACE keyspace1 WITH REPLICATION = { 'class' : 'SimpleStrategy', 'replication_factor' : 1 };
AlreadyExists: Keyspace 'keyspace1' already exists
cqlsh> █

# sqlsh
/bin/sh: 1: sqlsh: not found
# cqlsh
Connected to CassandraCluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.2 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> CREATE KEYSPACE keyspace2
... WITH REPLICATION = {
... 'class': 'SimpleStrategy',
... 'replication_factor': 2
... };
OperationTimeout: errors=['Connection defunct by heartbeat': 'Client request timeout. See Session.execute_async(timeout)', last_host=127.0.0.1:9042
cqlsh> CREATE KEYSPACE keyspace2 WITH REPLICATION = { 'class': 'SimpleStrategy', 'replication_factor': 2 };
AlreadyExists: Keyspace 'keyspace2' already exists
cqlsh> █

cqlsh> CREATE KEYSPACE keyspace1 WITH REPLICATION = { 'class' : 'SimpleStrategy', 'replication_factor' : 1 };
AlreadyExists: Keyspace 'keyspace1' already exists
cqlsh> CREATE KEYSPACE keyspace3
... WITH REPLICATION = {
... 'class': 'SimpleStrategy',
... 'replication_factor': 3
... };
cqlsh> DESCRIBE KEYSPACES;

keyspace1 keyspace3 system_auth system_schema system_views
keyspace2 system system_distributed system_traces system_virtual_schema
```

5. Спробуйте писати і читати в ці таблиці підключаюся на різні ноди.

```
cqlsh:keyspace2> SELECT * FROM items;
InvalidRequest: Error from server: code=2200 [Invalid query] message="table items does not exist"
cqlsh:keyspace2> CREATE TABLE keyspace1.orders ( order_id UUID PRIMARY KEY, customer_name TEXT, order_date TIMESTAMP, items LIST<
UUID>, total_amount DECIMAL );
AlreadyExists: Table 'keyspace1.orders' already exists
cqlsh:keyspace2> CREATE TABLE keyspace1.items ( id UUID PRIMARY KEY, name TEXT, category TEXT, price DECIMAL, manufacturer TE
XT, additional_properties MAP<TEXT, TEXT> );
AlreadyExists: Table 'keyspace1.items' already exists
cqlsh:keyspace2> CREATE TABLE keyspace2.items ( id UUID PRIMARY KEY, name TEXT, category TEXT, price DECIMAL, manufacturer TE
XT, additional_properties MAP<TEXT, TEXT> );
cqlsh:keyspace2> CREATE TABLE keyspace2.orders ( order_id UUID PRIMARY KEY, customer_name TEXT, order_date TIMESTAMP, items LIST<
UUID>, total_amount DECIMAL );
OperationTimeout: errors=['Connection defunct by heartbeat': 'Client request timeout. See Session.execute_async(timeout)', last_host=127.0.0.1:9042
cqlsh:keyspace2> CREATE TABLE keyspace2.orders ( order_id UUID PRIMARY KEY, customer_name TEXT, order_date TIMESTAMP, items LIST<
UUID>, total_amount DECIMAL );
AlreadyExists: Table 'keyspace2.orders' already exists
cqlsh:keyspace2> CREATE TABLE keyspace3.orders ( order_id UUID PRIMARY KEY, customer_name TEXT, order_date TIMESTAMP, items LIST<
UUID>, total_amount DECIMAL );
cqlsh:keyspace2> CREATE TABLE keyspace3.items ( id UUID PRIMARY KEY, name TEXT, category TEXT, price DECIMAL, manufacturer TE
XT, additional_properties MAP<TEXT, TEXT> );
OperationTimeout: errors=['Connection defunct by heartbeat': 'Client request timeout. See Session.execute_async(timeout)', last_host=127.0.0.1:9042
Warning: schema version mismatch detected; check the schema versions of your nodes in system.local and system.peers.

id | additional_properties | category | manufacturer | name | price
-----
6f6521a1-077c-4cc9-afdd-74f5cb1ba010 | null | Headphones | ASUS | Accesories | 100
(1 rows)
cqlsh:keyspace2>
cqlsh:keyspace2> INSERT INTO items (id, name, category, price, manufacturer) VALUES (uuid(), 'Jewelry', 'Gold Necklace', 9999, "Zoloto");
SyntaxException: line 1:121 no viable alternative at input ')' (... 'Jewelry', 'Gold Necklace', 9999, ["Zoloto"])...)
cqlsh:keyspace2> INSERT INTO items (id, name, category, price, manufacturer) VALUES (uuid(), 'Jewelry', 'Necklace', 9999, "Zoloto");
SyntaxException: line 1:116 no viable alternative at input ')' (... 'Jewelry', 'Necklace', 9999, ["Zoloto"])...)
cqlsh:keyspace2> INSERT INTO items (id, name, category, price, manufacturer) VALUES (uuid(), 'Jewelry', 'Necklace', 9999, "Zoloto");
SyntaxException: line 1:115 no viable alternative at input ')' (... 'Jewelry', 'Necklace', 9999, ["Zoloto"])...)
cqlsh:keyspace2> INSERT INTO items (id, name, category, price, manufacturer) VALUES (uuid(), 'Jewelry', 'Necklace', 9999, "Zoloto");
SyntaxException: line 1:115 no viable alternative at input ')' (... 'Jewelry', 'Necklace', 9999, ["Zoloto"])...)
cqlsh:keyspace2> INSERT INTO items (id, name, category, price, manufacturer) VALUES (uuid(), 'Jewelry', 'Necklace', 9999, "IDontKnow");
SyntaxException: line 1:118 no viable alternative at input ')' (... 'Jewelry', 'Necklace', 9999, ["IDontKnow"])...)
cqlsh:keyspace2> INSERT INTO items (id, name, category, price, manufacturer, additional_properties)
... VALUES (uuid(), 'Necklace', 'Jewelry', 9999, 'BrandName', {'material': 'gold', 'weight': '20g'});
cqlsh:keyspace2> SELECT * FROM items;

id | additional_properties | category | manufacturer | name | price
-----
6f6521a1-077c-4cc9-afdd-74f5cb1ba010 | null | Headphones | ASUS | Accesories | 100
bc62f21a-ad5d-4d30-a27c-0611bfdded73d | {'material': 'gold', 'weight': '20g'} | Jewelry | BrandName | NeckLace | 9999
```

```

cqlsh:keyspace3>
cqlsh:keyspace3> INSERT INTO items (id, name, category, price, manufacturer, additional_properties) VALUES (uuid(), 'Fork', 'Kitchen Stuff', 79, 'Forks ot Vlada', ());
cqlsh:keyspace3> SELECT * FROM items;

id | additional_properties | category | manufacturer | name | price
-----
ea619db5-0c3b-409f-9edb-d1ab2aa83216 | null | Kitchen Stuff | Forks ot Vlada | Fork | 79
0cf2b6fd-79fe-44c0-bb9b-d6fc9bfa10c9 | null | Kitchen Stuff | Chashki ot Vlada | Cup | 109

(2 rows)
cqlsh:keyspace3>

```

6. Вставте дані в створені таблиці і подивіться на їх розподіл по вузлах кластера для кожного з кейспесов (команда *nodetool status*)

```

id | additional_properties | category | manufacturer | name | price
-----
ea619db5-0c3b-409f-9edb-d1ab2aa83216 | null | Kitchen Stuff | Forks ot Vlada | Fork | 79
0cf2b6fd-79fe-44c0-bb9b-d6fc9bfa10c9 | null | Kitchen Stuff | Chashki ot Vlada | Cup | 109

(2 rows)
cqlsh:keyspace3> nodetool status
... ;
SyntaxException: line 1:0 no viable alternative at input 'nodetool' ([nodetool]...)
cqlsh:keyspace3> nodetool status;
SyntaxException: line 1:0 no viable alternative at input 'nodetool' ([nodetool]...)
cqlsh:keyspace3> exit
# nodetool status;

Datacenter: datacenter1
=====
Status=Up/Down
-- State=Normal/Leaving/Joining/Moving
-- Address Load Tokens Owns Host ID Rack
UN 172.19.0.4 255.38 KiB 16 ? 929fc404-7451-4ed1-b161-23d0f7586a9d rack1
UN 172.19.0.2 254.64 KiB 16 ? 14c7589a-d829-4a37-bb6c-8a2ef9824bb8 rack1
UN 172.19.0.3 279.19 KiB 16 ? 486a0aed-5f84-4dce-9576-35187ce21b18 rack1

Note: Non-system keyspaces don't have the same replication settings - effective ownership information is meaningless

```

7. Для якогось запису з кожного з кейспейсу виведіть ноди на яких зберігаються дані

```

cqlsh:keyspace1> USE keyspace1;
cqlsh:keyspace1> INSERT INTO items (id, name, category, price, manufacturer, additional_properties)
... VALUES (uuid(), 'Table', 'Furniture', 500, 'IKEA', {'color': 'brown'});
cqlsh:keyspace1>
cqlsh:keyspace1> USE keyspace2;
cqlsh:keyspace2> INSERT INTO items (id, name, category, price, manufacturer, additional_properties)
... VALUES (uuid(), 'Chair', 'Furniture', 100, 'IKEA', {'material': 'wood'});
cqlsh:keyspace2>
cqlsh:keyspace2> USE keyspace3;
cqlsh:keyspace3> INSERT INTO items (id, name, category, price, manufacturer, additional_properties)
... VALUES (uuid(), 'Lamp', 'Lighting', 150, 'Philips', {'power': '60W'});
cqlsh:keyspace3> SELECT id FROM items;

id
-----
ea619db5-0c3b-409f-9edb-d1ab2aa83216
6149fbf5-15fc-42e6-a39d-aa9900364072
0cf2b6fd-79fe-44c0-bb9b-d6fc9bfa10c9

(3 rows)
cqlsh:keyspace3> exit
# nodetool getendpoints keyspace3 items 0cf2b6fd-79fe-44c0-bb9b-d6fc9bfa10c9
172.19.0.2
172.19.0.4
172.19.0.3
#

```

8. Відключити одну з нод. Для кожного з кейспейсів перевірити з якими рівнями *consistency* можемо читати та писати

- для Keyspace з replication factor 1 - **CONSISTENCY ONE**
- для Keyspace з replication factor 2 - **CONSISTENCY ONE/TWO**

- для Keyspace 3 replication factor 3 - **CONSISTENCY ONE/TWO/THREE**

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	lab5v2	-	-	-	4.81%	2 hours ago	
☐	cassandra-node1	cb100f8a4c4f	cassandra:latest	9042:9042	0%	2 hours ago	
☐	cassandra-node3	53c6fe67caa1	cassandra:latest	9044:9042	2.33%	2 hours ago	
☐	cassandra-node2	80e8f53637a2	cassandra:latest	9043:9042	2.48%	2 hours ago	

```

Use HELP for help.
cqlsh> INSERT INTO keyspace3.items (id, name, category, price, manufacturer, additional_properties)
... VALUES (uuid(), 'Lamp', 'Lighting', 150, 'Philips', {'power': '60W'});
cqlsh> cqlsh -e "SELECT * FROM keyspace3.items WHERE id = <id>";
...
... ;
SyntaxException: line 1:0 no viable alternative at input 'cqlsh' ([cqlsh]...)
cqlsh> SELECT * From items;
InvalidRequest: Error from server: code=2200 [Invalid query] message="No keyspace has been specified. USE a keyspace.tablename"
cqlsh> use keyspace3;
cqlsh:keyspace3> use keyspace3;
cqlsh:keyspace3> SELECT * From items;

id | additional_properties | category | manufacturer | name | price
-----+-----+-----+-----+-----+-----
ea619db5-0c3b-409f-9edb-d1ab2aa83216 | null | Kitchen Stuff | Forks ot Vlada | Fork | 7
6149fbf5-15fc-42e6-a39d-aa9900364072 | {'power': '60W'} | Lighting | Philips | Lamp | 15
614fed19-feb3-40af-a4b2-a3bc15a6a731 | {'power': '60W'} | Lighting | Philips | Lamp | 15
0cf2b6fd-79fe-44c0-bb9b-d6fc9bfa10c9 | null | Kitchen Stuff | Chashki ot Vlada | Cup | 10

(4 rows)
cqlsh:keyspace3>

```

```

cqlsh:keyspace3> INSERT INTO keyspace1.items (id, name, category, price, manufacturer, additional_properties)
... VALUES (uuid(), 'Product', 'Category', 100, 'Manufacturer', {'property1': 'value1'});
NoHostAvailable: ('Unable to complete the operation against any hosts', {'Host: 127.0.0.1:9042 datacenter1: Unavailable('Error from server: code=1000 [Unavailable exception] message="Cannot achieve consistency level ONE" info={\'consistency\': \'ONE\', \'required_replicas\': 1, \'alive_replicas\': 0})'})
cqlsh:keyspace3> SELECT * From keyspace1.items;
NoHostAvailable: ('Unable to complete the operation against any hosts', {'Host: 127.0.0.1:9042 datacenter1: Unavailable('Error from server: code=1000 [Unavailable exception] message="Cannot achieve consistency level ONE" info={\'consistency\': \'ONE\', \'required_replicas\': 1, \'alive_replicas\': 0})'})
cqlsh:keyspace3>

```

Container CPU usage
7.17% / 800% (8 CPUs available)

Container memory usage
4.72GB / 7.09GB

Show charts

☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	lab5v2	-	-	-	7.17%	1 second ago	
☐	cassandra-node1	cb100f8a4c4f	cassandra:latest	9042:9042	0%	1 second ago	
☐	cassandra-node3	53c6fe67caa1	cassandra:latest	9044:9042	4.36%	2 hours ago	
☐	cassandra-node2	80e8f53637a2	cassandra:latest	9043:9042	2.81%	2 hours ago	

```
(4 rows)
cqlsh:keyspace3> INSERT INTO keyspace1.items (id, name, category, price, manufacturer, additional_properties)
... VALUES (uuid(), 'Product', 'Category', 100, 'Manufacturer', {'property1': 'value1'});
NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 datacenter1>: Unavailable('Error from server: code
=1000 [Unavailable exception] message="Cannot achieve consistency level ONE" info={\'consistency\': \'ONE\', \'required_replicas\': 1, \'alive_re
plicas\': 0})'})
cqlsh:keyspace3> SELECT * From keyspace1.items;
NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 datacenter1>: Unavailable('Error from server: code
=1000 [Unavailable exception] message="Cannot achieve consistency level ONE" info={\'consistency\': \'ONE\', \'required_replicas\': 1, \'alive_re
plicas\': 0})'})
cqlsh:keyspace3> INSERT INTO keyspace1.items (id, name, category, price, manufacturer, additional_properties) VALUES (uuid(), 'Product2', 'Categ
ory2', 102, 'Manufacturer2', {'property1': 'value1'});
cqlsh:keyspace3> SELECT * From keyspace1.items;
```

id	additional_properties	category	manufacturer	name	price
753b8c58-282f-4283-8688-6a5b49d659fd	{'color': 'brown'}	Furniture	IKEA	Table	500
89bf83a3-fb73-421c-a114-bfaf0b11f817	null	Electronics	Dell	Laptop	1200.50
f7bcf559-b1a8-4f48-8c3f-1cbd23be000c	null	Electronics	ASUS	Laptop	1200.50
75c6e73a-f027-43bc-aad1-cbc7b975ec5c	null	Electronics	Dell	Laptop	1200.50
7adaf0f1-cc24-490a-9c83-b2faefa80a80	{'property1': 'value1'}	Category2	Manufacturer2	Product2	102

```
(5 rows)
cqlsh:keyspace3>
```

9. Зробить так щоб три ноди працювали, але не бачили одна одну по мережі (заблокуйте чи відключити зв'язок між ними)

```
Project ▾
  ▾ lab5v2 ~/Documents/highload_syste
    ▾ .mvn
    ▾ cassandra_data
    ▾ src
      ▾ main
        ▾ java
          ▾ com.example.lab5v2
            ▾ models
              Item
              Order
            ▾ repositories
              ItemRepository
              OrderRepository
            ▾ services
              MainService
              Lab5v2Application
          ▾ resources
          ▾ test
          .gitattributes
          .gitignore
          docker-compose.yml
          HELP.md
          mvnw

lab5v2)
14 image: cassandra:latest
15 environment:
16   - CASSANDRA_SEEDS=cassandra-node2
17   command: ["-Dcassandra.auto_bootstrap=true", "-Dcassandra.storagedir=/var/lib/cassandra"]
18 volumes:
19   - ./cassandra_data:/var/lib/cassandra
20 networks:
21   - lab5v2_no-internal-communication
22
23 cassandra-node3:
24 image: cassandra:latest
25 environment:
26   - CASSANDRA_SEEDS=cassandra-node3
27   command: ["-Dcassandra.auto_bootstrap=true", "-Dcassandra.storagedir=/var/lib/cassandra"]
28 volumes:
29   - ./cassandra_data:/var/lib/cassandra
30 networks:
31   - lab5v2_no-internal-communication
32
33 networks:
34   lab5v2_no-internal-communication:
35     driver: bridge
36
```

```
Terminal Local ▾
WARN[0000] /Users/prosto/Documents/highload_systems/lab5v2/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid confusion
[*] Running 4/4
✓ Network lab5v2_lab5v2_no-internal-communication Created
✓ Container lab5v2-cassandra-node3-1 Started
✓ Container lab5v2-cassandra-node2-1 Started
✓ Container lab5v2-cassandra-node1-1 Started
(base) prosto@MacBook-Pro-Malchik lab5v2 %
```

```
PS E:\git\highload-tasks\Cassandra> docker network disconnect cassandra_cassandra-net cassandra-2
PS E:\git\highload-tasks\Cassandra> docker network disconnect cassandra_cassandra-net cassandra-3
PS E:\git\highload-tasks\Cassandra>
```

10. Для кейспейсу з *replication factor* 3 задайте рівень consistency рівним 1. Виконайте по черзі запис значення з однаковим primary key, але різними іншими значенням окремо на кожну з нод (тобто створіть конфлікт)

```
cqlsh> use repl_three
...
... ;
cqlsh:repl_three> INSERT INTO items(id, name) VALUES (101, 'three_item_two')
... ;
cqlsh:repl_three> 
```

зробив таку операцію на 3 різних нодах

11. Відновіть зв'язок між нодами, і перевірте що вони знову об'єдналися у кластер. Визначте яким чином була вирішений конфлікт даних та яке значення було прийнято кластером та за яким принципом

```
PS E:\git\highload-tasks\Cassandra> docker network connect cassandra_cassandra-net cassandra-2
PS E:\git\highload-tasks\Cassandra> docker network connect cassandra_cassandra-net cassandra-3
```

```
cqlsh:repl_three> select * from items;
```

id	name
5	three_item_5
10	three_item_10
11	three_item_11
1	three_item_1
8	three_item_8
2	three_item_2
4	three_item_4
7	three_item_7
6	three_item_6
9	three_item_9
101	three_item_three
12	three_item_12
3	three_item_3

ВИСНОВОК

Виконавши дану лабораторну роботу, я дослідив основи моделювання даних у Cassandra та особливості роботи з базами даних типу column family. Я дійшов до висновку, що ефективне моделювання даних у Cassandra потребує ретельного планування щодо використання ключових просторів, таблиць, карт та списків. Це дозволяє забезпечити швидкий доступ до даних навіть при масштабуванні на великі кластери.

SimpleStrategy: Використовується для однорідних кластерів, коли всі вузли однакові. Ця стратегія дозволяє встановити певний рівень реплікації (1, 2, 3) для забезпечення доступності даних у разі відмови вузлів.

Ефективне використання реплікації дозволяє підтримувати високу доступність даних, швидкість обробки запитів і мінімізувати можливі втрати даних у разі відмови вузлів у кластері. Важливо ретельно налаштовувати реплікацію, враховуючи специфіку застосування та вимоги до продуктивності та доступності.